TRYSTOP

Complete Developer Handover & Integration Specification

Detailed Technical Architecture, Step-by-Step User & Administrative Application Flows, API Endpoints, Dynamic Filtering Schemes, Database Models, and Production Configurations

---

Version 1.1.0 — Hyperlocal Fashion E-Commerce Platform
Confidential System Handover Document
Generated on July 2026

Table of Contents

1. Detailed User App Flow & API Sequences

This section details the step-by-step user shopping journey from starting the application to viewing product specifications, highlighting API invocation details, query parameters, state management, and the hyperlocal stock checking logic.

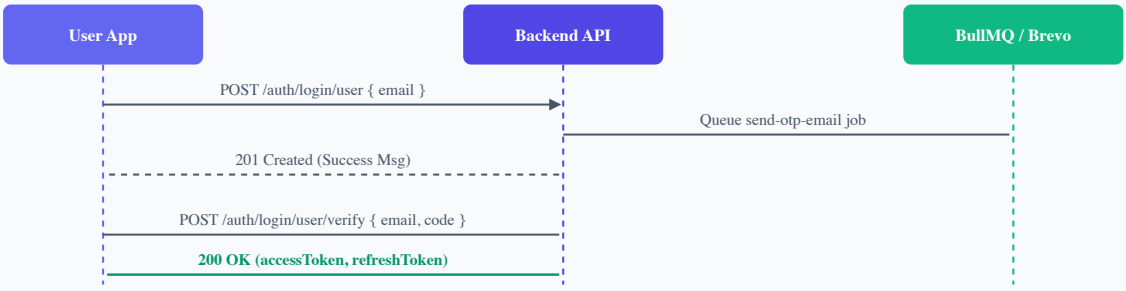
1.1 App Bootstrapping & Authentication Flow

When the client app starts, it checks local storage for an active session to skip the login screen:

1. **Check Tokens:** The app inspects its secure storage for an `accessToken` and a `refreshToken`.
2. **Token Validation:** If an access token exists, the client decodes it locally to verify the expiration time. If valid, the user immediately enters the Home screen.
3. **Token Refresh:** If the access token has expired but a refresh token is present, the app calls:
   `POST /auth/refresh`
   Headers: None required (passed in request body).
   Payload: `{ "refreshToken": "eyJhbGciOi..." }`
   Success Response (200 OK): Returns a new `accessToken` and `refreshToken`. The app stores these locally and proceeds to the Home screen.
4. **Navigate to Login:** If no tokens are present or the refresh token is invalid/expired (returning a 401 Unauthorized), the client displays the User Login screen.

1.1.1 Passwordless Login Flow (User App Perspective)

Authentication for end-users uses email-based passwordless OTP validation.



Step-by-step Request Execution:

- **API Call 1 (Send OTP Request):**
  `POST /auth/login/user`
  Payload: `{ "identifier": "customer@email.com" }`
  Backend Process: Generates a random 6-digit numeric string (e.g. `428913`). Stores a cryptographically hashed version of the code mapped to the email identifier in the database `Otp` collection with a 5-minute TTL. Publishes an asynchronous task (`send-otp-email`) containing the email and code payload to the Redis-backed BullMQ queue.
  Response: `201 Created` is returned immediately (typically under 30ms).

- **Background Task Processing:** The BullMQ background processor consumes the queued task, communicates with the Brevo API, and dispatches the verification code to the customer.
- **API Call 2 (Validate Verification Code):**
  `POST /auth/login/user/verify`
  Payload: `{ "identifier": "customer@email.com", "code": "428913" }`
  Backend Process: Searches the database for the active OTP mapped to the email. If the record exists, is unexpired, and matching:
  - Sets the OTP record's status to prevent code re-use.
  - Checks if a profile already exists in the `users` database. If the email is new, it automatically registers a new account: `{ email: "customer@email.com", role: "user", isActive: true }`.
  - Issues a pair of JWT tokens (AccessToken valid for 15 minutes, RefreshToken valid for 7 days).

  Response: `200 OK` returning the generated tokens and user profile details.

1.2 Home Screen Rendering (Home BFF Aggregation)

Once logged in, the client app requests the dynamic homepage BFF aggregator. Rather than executing multiple network roundtrips to fetch banners, category grids, and individual carousels, the client executes a single endpoint query:
`GET /home`
Authentication Header: None required (guest-accessible endpoint).
Caching: Cached globally in Redis for 30 seconds to handle high-traffic peaks.

1.2.1 Understanding the Aggregated Structure

The API returns an ordered list of sections configured by system administrators. Each section dictates what content and style the mobile client should render:

```
[
  {
    "_id": "603d2b...",
    "type": "banner_carousel",
    "style": "banner_full",
    "order": 1,
    "data": [
      {
        "_id": "60a1d4...",
        "imageUrl": "https://cloudinary.com/.../summer_sale.jpg",
        "clickAction": { "type": "category", "target": "men-shirts" }
      }
    ]
  },
  {
    "_id": "603d2c...",
    "type": "category_grid",
    "style": "grid_2col",
    "title": "Shop by Category",
```

```
    "order": 2,
    "data": [
      { "_id": "603a11...", "name": "Men", "slug": "men", "parentCategoryId": null },
      { "_id": "603a12...", "name": "Women", "slug": "women", "parentCategoryId": null }
    ]
  },
  {
    "_id": "603d2d...",
    "type": "product_carousel",
    "style": "strip",
    "title": "Trending Now",
    "order": 3,
    "data": [
      { "_id": "607e15...", "name": "Oversized Fit Tee", "brand": "TryStop", "offerPrice": 999 }
    ]
  }
]
```

2. Server-Driven UI & Dynamic Filter Panel Logic

Trystop implements a **Server-Driven UI (SDUI)** architecture for both its homepage sections and search filters. This structure allows administrators to adjust layouts and modify available filtering criteria directly from the Admin Panel database without releasing updates to the app stores.

2.1 Homepage Section Styles & Client Component Mapping

When the client app parses the response from `GET /home`, it maps the section `type` and `style` fields directly to native UI components:

Section Type	Style Parameter	Expected UI Layout & Actions
`banner_carousel`	`banner_full`	Renders a horizontal image slider with paginating indicators. Each image contains a touch handler that navigates the user based on the `clickAction.type` (e.g., category, product details).
`category_grid`	`grid_2col` / `grid_3col`	Displays category icons in a multi-column grid. Tapping an item navigates the user to the category search results page.
`product_carousel`	`strip`	Renders a horizontally scrollable product carousel. Product cards display the image, brand, title, offer price, and discount percentage badge.
`deal_strip`	`deal_timer`	Renders a promotional section featuring product cards alongside a countdown timer. The timer counts down to zero, indicating the end of the promotional period.

2.2 Database-Driven Dynamic Search Filter Panel

When users navigate to a product list or search page, the client app requests the dynamic filter configurations:
`GET /filters`
Query Parameters: `category=men` (Optional: filters options to show only criteria applicable to that category).
Success Response (200 OK):

```
[
  {
    "_id": "603f7a...",
    "key": "color",
    "label": "Color",
    "widget": "swatch",
    "options": [
      { "value": "black", "label": "Black", "hex": "#000000" },
      { "value": "white", "label": "White", "hex": "#FFFFFF" }
    ]
  },
  {
```

```
    "_id": "603f7b...",
    "key": "size",
    "label": "Size",
    "widget": "chips",
    "options": [
      { "value": "M", "label": "M" },
      { "value": "L", "label": "L" }
    ]
  },
  {
    "_id": "603f7c...",
    "key": "price",
    "label": "Price",
    "widget": "range",
    "min": 0,
    "max": 5000
  }
]
```

2.2.1 Frontend Dynamic Widget Rendering & Selection State

The client app iterates through the returned filters array and renders a custom UI control based on the `widget` field:

- **widget: "swatch"** → Renders a grid of circular color buttons filled with the corresponding `hex` value. When a color is selected, the application appends the value to the search query parameters: `color=black`.
- **widget: "chips"** → Renders a row of text chips (e.g. size buttons, fit styles). Tapping a chip updates the query parameters: `size=M`.
- **widget: "range"** → Renders a slider component using the `min` and `max` values. Adjusting the slider updates the query parameters: `priceMin=500&priceMax=2000`.

2.3 Hyperlocal Stock & PDP Verification Logic

hyperlocal fast delivery is verified during both the search results display and on the Product Detail Page (PDP):

1. **Define User Location / Hub ID:** When the app launches, it resolves the user's location via GPS and requests the closest delivery hub ID (e.g. `hub_west_01`).
2. **Query Paginated Products:** When browsing categories or search results, the client app appends the hub ID to the search query:
   `GET /products?subcategory=shirts&hubId=hub_west_01`
   Backend Execution: The backend queries the database for active shirts and inspects the `variants.stockByHub` array. It filters results to only return products with `quantity > 0` at the specified `hubId`, ensuring users only see items available for immediate delivery.
3. **PDP Verification (GET /products/:id):** On the PDP, the client app verifies stock levels for the selected size and color variant:

```
// Example PDP Response item variants:
"variants": [
  {
    "size": "L",
    "color": "Navy Blue",
    "sku": "TS-NAVY-L",
    "stockByHub": [
      { "hubId": "hub_west_01", "quantity": 12 },
      { "hubId": "hub_east_02", "quantity": 0 }
    ]
  }
]
```

If the user's current hub is `hub_west_01`, the client app displays "In Stock (Delivery in 2 hours)". If the hub is `hub_east_02`, the app disables the "Buy Now" button and displays "Out of Stock in your area".

3. Admin & Sub-Admin Backoffice Management

The Admin Panel provides tools for administrators to manage the catalog, configure homepage sections, and review products submitted by sellers.

3.1 Administrative Access Bootstrapping

To bootstrap the administrative layer on a new deployment, the system exposes a setup endpoint:
`POST /admin-auth/bootstrap`
Payload: `{ "email": "admin@trystop.com", "password": "SecurePassword123" }`
Backend Protection: This endpoint can only be run once. The service counts existing documents in the `Admin` collection; if the database is not empty, the endpoint throws a `403 Forbidden` exception.

3.2 Sub-admin Management & Permissions Assignment

Super Admins can create sub-admins and assign specific permission scopes:
`POST /admin-auth/create-subadmin`
Headers: `Authorization: Bearer`
Payload:

```
{
  "email": "subadmin_moderator@trystop.com",
  "password": "TempPassword999",
  "permissions": ["approve_products", "manage_catalog"]
}
```

These permission strings map directly to route protection decorators:
`@RequirePermission('approve_products')`

3.3 Admin Homepage Section Configuration & BFF Sync

Administrators configure homepage sections using the following endpoints:

- **Create New Layout Section:**
  `POST /home/sections`
  Payload:

```
{
  "type": "product_carousel",
  "title": "Monsoon Specials",
  "style": "strip",
  "order": 5,
  "filter": {
    "tag": "monsoon_sale",
```

```
    "priceMax": 1500
  }
}
```

- **Update Layout Section:**
  `PATCH /home/sections/:id`
- **Delete Layout Section:**
  `DELETE /home/sections/:id`

Behind the Scenes BFF Sync Lifecycle:

3.4 Product Moderation Gate & Review Workflow

Products added by sellers must go through a review queue before appearing in public searches or homepage carousels.

- **Approve Listing:**
  `PATCH /products/:id/approve`
  Backend Execution: Updates the product's status to `live`. This immediately invalidates the cached product details in Redis and makes the product available for indexing and display in user search results.
- **Reject Listing:**
  `PATCH /products/:id/reject`
  Payload: `{ "reason": "Incorrect category specifications, neckline attribute must be set" }`
  Backend Execution: Sets the product status to `rejected` and stores the rejection reason. The product remains hidden from public endpoints but is visible in the seller's catalog portal for modification.
- **Assign Promotional Tags:**
  `PATCH /products/:id/tags`
  Payload: `{ "tags": ["trending", "monsoon_sale"] }`
  Backend Execution: Assigns the specified tags to the product. The product will now appear in any homepage section filtered by those tags.

4. Seller Catalog Onboarding & Media Flow

This section details how sellers complete registration, upload product media, fetch dynamic category attribute requirements, and submit new product listings.

4.1 Seller Account Creation

Sellers register using email and password credentials.
`POST /auth/register/seller`
Payload:

```
{
  "email": "seller@company.com",
  "phone": "+919876543210",
  "password": "StrongPasswordPass1",
  "businessName": "Express Apparels",
  "businessAddress": "Hub 4, Outer Road, Delhi",
  "gstin": "07AAAAA1111A1Z1"
}
```

Once registered, the account remains in a `pending` status until an administrator verifies the business credentials and GSTIN. After verification, the administrator updates the seller's status to `active`, enabling catalog creation.

4.2 Media Upload Flow

Trystop requires product images and video demonstrations to be uploaded to Cloudinary via a stream-based backend endpoint.

1. **Image Upload Request:**
   `POST /upload/image`
   Request Type: Multipart Form Data
   Key: `file` (Contains raw binary file data).
   Backend Processing: Validates that the file size is under 5MB and that the file format is an image (e.g. JPEG, PNG, WEBP). Streams the file to the Cloudinary service, applying automatic quality optimization (`quality: auto, fetch_format: auto`).
   Success Response (200 OK):

   ```
   {
     "message": "Image uploaded successfully",
     "url": "https://res.cloudinary.com/.../trystop_users/summer_tee.webp",
     "publicId": "trystop_users/summer_tee",
     "bytes": 284102
   }
   ```

2. **Saving Image URL:** The client app stores the returned `url` and sends it in the product creation payload.

4.3 Fetching Category Attribute Templates

To show a product upload form with fields relevant to the chosen subcategory (e.g., Rise and Stretchable fields for Jeans, Sleeve and Neckline fields for T-Shirts), the seller app calls:
`GET /categories/:subcategoryId/attributes`

Response Attribute List:

```
{
  "subcategoryId": "603d2c...",
  "fields": [
    { "name": "Fit", "type": "select", "options": ["Slim", "Regular", "Oversized"], "required": tr
    { "name": "Fabric", "type": "text", "options": [], "required": true },
    { "name": "Pockets", "type": "boolean", "options": [], "required": false }
  ]
}
```

The seller app renders the upload form dynamically based on this list. When submitting the product creation request (`POST /products`), the form data is mapped to the product's `specifications` object:
`"specifications": { "Fit": "Slim", "Fabric": "90% Cotton", "Pockets": true }`

5. Rider Delivery Verification & On-Duty Lifecycle

Riders use OTP-based authentication. They must submit documentation for verification and update their status (online/offline) to receive delivery requests.

5.1 Rider Registration & Verification Lifecycle

Riders register by submitting their personal details, vehicle type, and driver's license details:
`POST /auth/register/rider`
Payload:

```
{
  "email": "rider_john@email.com",
  "phone": "+918888888888",
  "name": "John Doe",
  "vehicleType": "bike",
  "licenseNumber": "DL-1420230008892",
  "licenseImage": "https://res.cloudinary.com/trystop/image/upload/license_placeholder.jpg"
}
```

Upon registration, the rider profile is created with a `pending` status. Administrators review the driver's license and vehicle details before activating the rider.

5.2 On-Duty Status Switch

To receive hyperlocal delivery requests, verified riders toggle their availability status:
`PATCH /auth/rider/status`
Headers: `Authorization: Bearer`
Payload: `{ "isOnline": true }`
Backend Execution: Updates the rider's record in the database. When a new order is dispatched from a hub, the logistics engine filters for riders in the area who are currently active and online (`status: 'active', isOnline: true`).

5.3 Rider Login Sequence Diagram

6. Complete Database Models & Indexes Reference

The primary data models are implemented using Mongoose schemas.

6.1 Product Schema

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { Document, Types } from 'mongoose';

@Schema({ timestamps: true })
export class Product {
  @Prop({ required: true })
  name: string;

  @Prop({ required: true, unique: true })
  slug: string;

  @Prop({ required: true })
  description: string;

  @Prop({ required: true })
  brand: string;

  @Prop({ type: Types.ObjectId, ref: 'Category', required: true })
  categoryId: Types.ObjectId;

  @Prop({ type: Types.ObjectId, ref: 'Category', required: true })
  subcategoryId: Types.ObjectId;

  @Prop({ required: true, enum: ['men', 'women', 'kids', 'unisex'] })
  gender: string;

  @Prop({ required: true })
  mrp: number;

  @Prop({ required: true })
  offerPrice: number;

  @Prop({ default: 0 })
  discountPercent: number;

  @Prop({
    type: [{
      size: { type: String, required: true },
      color: { type: String, required: true },
```

```
    colorHex: { type: String },
    sku: { type: String, required: true },
    stockByHub: [{
      hubId: { type: String, required: true },
      quantity: { type: Number, required: true, default: 0 }
    }]
  }],
  required: true
})
variants: Array<{
  size: string;
  color: string;
  colorHex?: string;
  sku: string;
  stockByHub: Array<{ hubId: string; quantity: number }>;
}>;

@Prop({ type: Object, default: {} })
specifications: Record;

@Prop({ type: [String], default: [] })
images: string[];

@Prop({ type: { type: String } }) // Union type fix
video?: string;

@Prop({ default: true })
isReturnable: boolean;

@Prop({ default: 7 })
returnWindowDays: number;

@Prop({ default: false })
isApproved: boolean;

@Prop({
  required: true,
  enum: ['pending_review', 'live', 'rejected', 'deleted'],
  default: 'pending_review'
})
status: string;

@Prop({ type: { type: String } }) // Union type fix
rejectionReason?: string;

@Prop({ type: [String], default: [] })
tags: string[];
```

```
  @Prop({ type: Types.ObjectId, required: true })
  uploadedBy: Types.ObjectId;

  @Prop({ required: true, enum: ['seller', 'superadmin', 'subadmin'] })
  uploadedByRole: string;
}
```

6.2 Category Schema

```
@Schema({ timestamps: true })
export class Category {
  @Prop({ required: true })
  name: string;

  @Prop({ required: true, unique: true })
  slug: string;

  @Prop({ type: Types.ObjectId, ref: 'Category', default: null })
  parentCategoryId: Types.ObjectId | null;

  @Prop({ default: '' })
  icon: string;

  @Prop({ default: true })
  isActive: boolean;
}
```

6.3 Dynamic Filter Option Schema

```
@Schema({ timestamps: true })
export class FilterOption {
  @Prop({ required: true, unique: true })
  key: string;

  @Prop({ required: true })
  label: string;

  @Prop({ required: true, enum: ['swatch', 'chips', 'range', 'multiselect'] })
  widget: string;

  @Prop({
    type: [{
      value: { type: String, required: true },
      label: { type: String, required: true },
```

```
      hex: { type: String }
    }],
    default: []
  })
  options: Array<{ value: string; label: string; hex?: string }>;

  @Prop()
  min?: number;

  @Prop()
  max?: number;

  @Prop({ type: [String], default: [] })
  applicableCategories: string[];
}
```

7. Production API Reference Specifications

7.1 Public Catalog Endpoints

7.1.1 GET /products

Returns a list of approved products matching the provided filters. Used by the search and browse screens.

- **Authentication Required:** No
- **Headers:** `Content-Type: application/json`
- **Query Parameters:**

Parameter	Type	Example	Description
`category`	String	`men`	Top-level category slug.
`subcategory`	String	`t-shirts`	Leaf subcategory slug.
`search`	String	`blue shirt`	Full-text search query.
`hubId`	String	`hub_west_01`	Hyperlocal hub filter.
`limit`	Number	`20`	Page size.

- **Success Response (200 OK):**

```
{
  "products": [
    {
      "_id": "607e15...",
      "name": "Oversized Fit Cotton Shirt",
      "brand": "TryStop",
      "offerPrice": 1299,
      "images": ["https://res.cloudinary.com/.../img1.webp"],
      "discountPercent": 35
    }
  ],
  "nextCursor": "eyJsYXN0SWQiOi..."
}
```

7.2 User Authentication Endpoints

7.2.1 POST /auth/login/user

Initiates passwordless login by sending an OTP to the user's email.

- **Authentication Required:** No
- **Request Body:**

```
{
  "identifier": "customer@email.com"
}
```

- **Success Response (201 Created):**

```
{
  "message": "OTP verification code sent successfully to email"
}
```

7.2.2 POST /auth/login/user/verify

Verifies the OTP code sent to the user's email and returns session tokens.

- **Authentication Required:** No
- **Request Body:**

```
{
  "identifier": "customer@email.com",
  "code": "428913"
}
```

- **Success Response (200 OK):**

```
{
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi...",
  "user": {
    "id": "603d2b...",
    "email": "customer@email.com",
    "role": "user"
  }
}
```

8. Infrastructure Optimization: Redis & BullMQ Queues

The platform uses caching and background job processing to handle high traffic and reduce request latency.

8.1 Redis Cache Policies

Read-heavy public endpoints are cached using Redis, with cache invalidation triggered automatically when underlying data changes:

Endpoint	Cache Key	TTL (Time-To-Live)	Invalidation Hook
`GET /home`	`home:page`	30 Seconds	Invalidated when an administrator updates the home sections (`PATCH /home/sections/:id`).
`GET /categories`	`categories:tree`	60 Seconds	Invalidated when categories are created, updated, or deleted.
`GET /products/:id`	`product:detail:`	15 Seconds	Invalidated when an administrator approves changes or updates stock values.

8.2 BullMQ Background Processing Queue

Asynchronous jobs are processed out-of-process using a Redis-backed BullMQ queue. For example, OTP notifications are processed by a dedicated consumer:

```
1. User Requests OTP → App publishes a job to the "notification" queue.
2. HTTP thread immediately returns a 201 response.
3. NotificationProcessor consumes the job from Redis:
   @Process('send-otp-email')
   async handleSendOtpEmail(job: Job) {
     await this.notificationService.sendOtpViaEmailDirect(job.data.email, job.data.otp);
   }
4. Job failures are automatically retried by BullMQ up to 3 times with exponential backoff.
```